Optimus-Cost-Agent

Low-Level Design (LLD) Specification

Gateway-Centric Cost Governance, Tooling, Observability and Runtime Control

Version 2.39

Architected by: Vibhanshu Agarwal

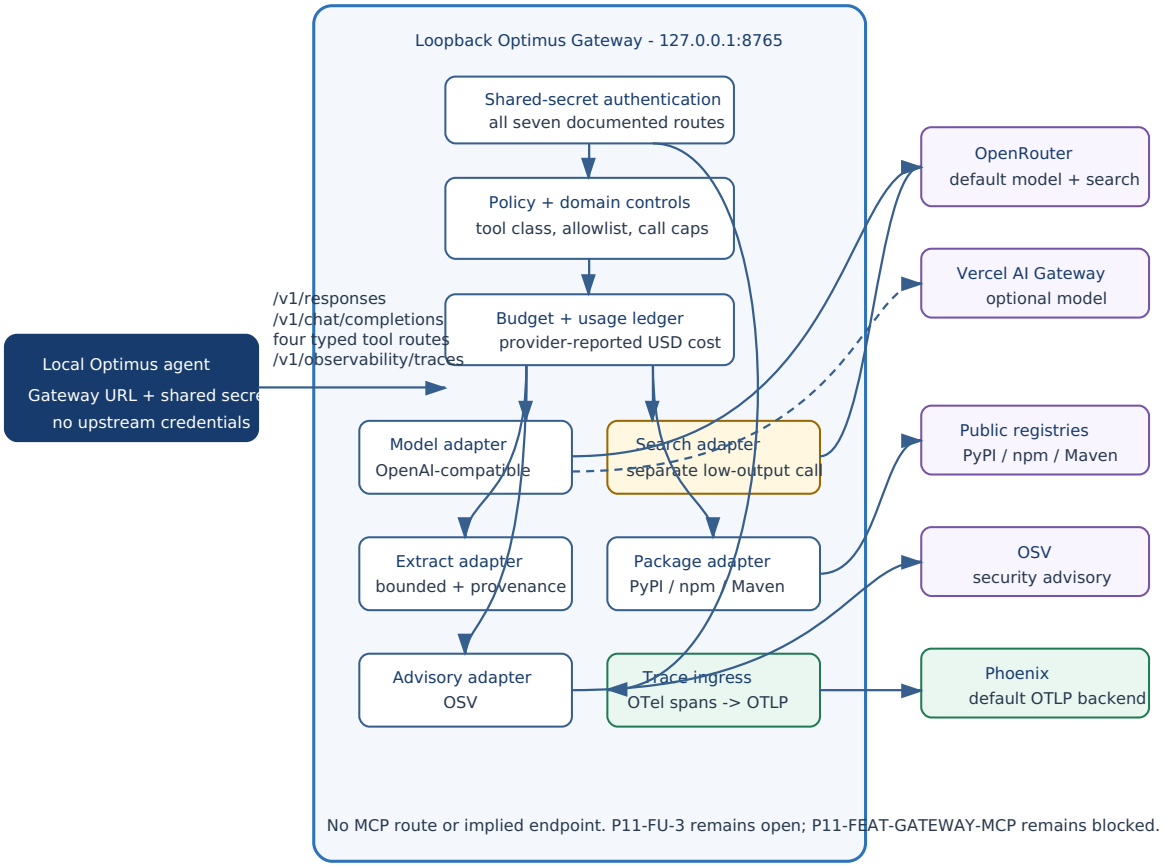
0. Optimus AI Gateway Architecture

The Optimus Gateway is a deterministic local process bound to loopback. It separates the LLM-driven agent from upstream credentials, policy enforcement, budget authority, provider usage normalization, and controlled external egress.

The agent process receives only `OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY`. The Gateway process holds one developer-owned aggregator credential in its own environment or OS credential storage. There is no hosted Optimus service, OAuth/device flow, tenant wallet, Vault, or public Gateway origin.

A. Recommended architecture

OpenRouter is the default aggregator. Vercel AI Gateway is an optional second OpenAI-compatible model endpoint when Python integration remains modest. Direct single-provider adapters are removed.



Complete local Gateway component flow

The complete route flow fits on the page. No MCP endpoint is shown or implied.

C. Gateway responsibilities

- Authenticate the local agent with a shared secret on strict loopback.
- Resolve model aliases through an approved OpenAI-compatible aggregator.
- Broker deterministic search, bounded extract, package lookup, and OSV advisory independently.
- Enforce execution mode, tool policy, domain rules, call caps, provenance, and run budget.
- Persist run/session/request/Gateway/provider request attribution.
- Isolate upstream credentials from the agent process.
- Accept structured trace ingress, map it to OTel, and export OTLP.

D. Gateway-facing API shape

```
# Responses API shape - top-level "input"
POST /v1/responses

# Chat Completions shape - "messages" array
POST /v1/chat/completions

# Typed tools
POST /v1/tools/web/search
POST /v1/tools/web/extract
POST /v1/tools/package/lookup
POST /v1/tools/security/advisory

# Authenticated structured trace ingress
POST /v1/observability/traces
```

All routes authenticate with the local shared secret. Trace ingress is operational, not a model/tool billing route. No MCP endpoint is added.

E. Process-scoped configuration boundary

Agent process	Gateway process
`OPTIMUS_GATEWAY_URL`	`OPTIMUS_LOCAL_GATEWAY_PROVIDER=openrouter`
`OPTIMUS_API_KEY`	`OPTIMUS_LOCAL_GATEWAY_PROVIDER_API_KEY`
No upstream or OTel key	`OPTIMUS_LOCAL_GATEWAY_SHARED_SECRET`
Strict loopback URL	domains, Redis URL, optional OTLP endpoint
No override surface	no hosted origin, tenant profile, or production mode

0A. Local vs. Gateway Configuration and Provider-Cost Mapping

Phase 1 runtime rule

The agent runtime allows only `OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY`. The Gateway owns the developer's approved aggregator credential. Local agent-side Tavily, OpenAI, OpenRouter, GLM, LangSmith, or other provider keys are rejected.

Provider-cost mapping

`OPTIMUS_API_KEY` is a local shared secret, not a wallet key. The developer funds the aggregator account directly. The Gateway records the aggregator response into one local ledger.

Call class	Required accounting
Model completion	aggregator, actual provider/model when returned, tokens/cache, billing units, provider-reported `cost_usd`, request IDs
Deterministic search	separate minimal model call, structured citations, search-use accounting, provider-reported `cost_usd`
Package/advisory	operational request evidence; no fabricated provider cost
Trace export	delivery state and trace IDs; no allocated/amortized request charge

```
Agent shared secret
  -> loopback Gateway policy and budget
  -> developer aggregator account
  -> provider-reported usage and USD cost
  -> GatewayUsage + normalized local ledger
```

The separate USD rename removes `optimus_credits_debited` and other legacy credit-named fields without changing their existing USD semantics or introducing cross-run policy.

Gateway Tool and Observability Endpoints

The agent never calls OpenRouter, Vercel, package registries, OSV, Phoenix, or another OTLP backend directly. It calls the loopback Gateway's typed endpoints.

```
POST /v1/responses
POST /v1/chat/completions
POST /v1/tools/web/search
POST /v1/tools/web/extract
POST /v1/tools/package/lookup
POST /v1/tools/security/advisory
POST /v1/observability/traces
```

Search is an authorized, separate, minimal OpenRouter plugin request. Package and advisory routes remain independent of the paid-search configuration.

The canonical observability path is:

```
agent structured trace event
  -> authenticated /v1/observability/traces
  -> Gateway validation and redaction
  -> OpenTelemetry span/event mapping
  -> OTLP export
  -> Phoenix by default
```

The agent has no backend credential or direct backend egress. LangSmith and amortized observability-cost accounting are not part of this architecture. The route list has no MCP endpoint.

1. ACP Protocol Framing & JSON-RPC Contract Specification

To prevent streaming fragmentation errors, the ACP interface enforces a strict HTTP-style byte framing header layout. Malformed payloads trigger immediate JSON-RPC standard error blocks.

A. Header, Buffer Size Limits & Serialization Standards

Every frame must prepend an explicit Content-Length prefix sequence. To prevent Denial of Service (DoS) attacks, the transport plane enforces a strict Maximum Header Size of 8 KB and a Maximum Body Size of 10 MB. Responses echo a matching structural correlation integer via the id field.

```
[INBOUND SYSTEM STREAM TRANSACTION]
Content-Length: 114

{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "agent/prompt",
  "params": {
    "prompt": "Refactor...",
    "targetFile": "main.py"
  }
}
```

```
PARSE_ERROR = {
    "code": -32700,
    "message": "Parse error: Invalid JSON payload received."
}
INVALID_REQUEST = {
    "code": -32600,
    "message": "Invalid Request: Schema mapping parameters missing."
}
INTERNAL_ERROR = {
    "code": -32603,
    "message": "Internal error: Underlying task execution failure."
}
ID_COLLISION_ERROR = {
    "code": -32001,
    "message": "Server error: Request ID collision detected."
}
```

2. Cross-Platform Stream Transport Layer

```
import asyncio
import sys
import json
import re
from typing import Optional, Tuple
from json import JSONDecodeError
from concurrent.futures import ThreadPoolExecutor

class CrossPlatformStreamReader:
    def __init__(self, max_header_bytes: int = 8192,
                 max_body_bytes: int = 10485760):
        self.executor = ThreadPoolExecutor(max_workers=1)
        self.max_header = max_header_bytes
        self.max_body = max_body_bytes

    def _sync_blocking_read(self) -> Tuple[Optional[bytes], bool]:
        header = b""
        while b"\r\n\r\n" not in header:
            if len(header) >= self.max_header:
                return None, True
            char = sys.stdin.buffer.read(1)
            if not char:
                return b"", False
            header += char
        match = re.search(b"(?i)Content-Length\s*:\s*(\d+)", header)
        if not match or int(match.group(1)) > self.max_body:
            return None, True
        content_length = int(match.group(1))
        return sys.stdin.buffer.read(content_length), False

    async def read_frame(self) -> Tuple[Optional[dict], Optional[dict]]:
        loop = asyncio.get_running_loop()
        raw_bytes, bad_frame = await loop.run_in_executor(
            self.executor, self._sync_blocking_read
        )
        if bad_frame:
            return None, PARSE_ERROR
        if not raw_bytes:
            return None, None
        try:
            return json.loads(raw_bytes.decode("utf-8")), None
        except JSONDecodeError:
            return None, PARSE_ERROR
```

3. Queued Task Lifecycle, Backpressure Controls & Runtime Pooling

```
import httpx
import uuid
import redis.asyncio as aioredis

class ManagedTaskManager:
    def __init__(self, max_concurrent_prompts: int = 3,
                 execution_timeout_secs: int = 120):
        self.semaphore = asyncio.Semaphore(max_concurrent_prompts)
        self.timeout = execution_timeout_secs
        self.all_tracked_tasks = {}
        self.http_pool = None
        self.redis_pool = None

    async def initialize_runtime_pools(self, redis_url: str):
        if self.all_tracked_tasks:
            raise RuntimeError(
                "Cannot reinitialize pools while tasks are running."
            )
        if self.http_pool or self.redis_pool:
            if self.http_pool:
                await self.http_pool.aclose()
            if self.redis_pool:
                await self.redis_pool.disconnect()
        self.http_pool = httpx.AsyncClient(
            limits=httpx.Limits(
                max_connections=10, max_keepalive_connections=5
            ),
            timeout=httpx.Timeout(30.0),
        )
        self.redis_pool = aioredis.ConnectionPool.from_url(
            redis_url, max_connections=20, decode_responses=True
        )
```

```
    async def enqueue_and_execute(self, request_id: Optional[int],
                                  pipeline_coro_factory) -> dict:
        active_id = request_id if request_id is not None \
            else int(uuid.uuid4().int & 0x7FFFFFFF)
        if not self.http_pool or not self.redis_pool:
            return {
                "status": "ERROR",
                "error": {
                    "code": -32003,
                    "message": "Server error: Connection pools uninitialized.",
                },
            }
        if active_id in self.all_tracked_tasks:
            return {
                "status": "ERROR",
                "error": {
                    "code": -32001,
                    "message": "Server error: Request ID collision detected.",
                },
            }

        async def _orchestrated_run():
            async with self.semaphore:
                coro = pipeline_coro_factory(self.http_pool, self.redis_pool)
                return await asyncio.wait_for(coro, timeout=self.timeout)

        loop = asyncio.get_running_loop()
        wrapper_task = loop.create_task(_orchestrated_run())
        self.all_tracked_tasks[active_id] = wrapper_task
        try:
            result = await wrapper_task
            return {"status": "SUCCESS", "data": result}
        except asyncio.CancelledError:
            return {
                "status": "ERROR",
                "error": {"code": -32603, "message": "Task cancelled during shutdown."},
            }
        except asyncio.TimeoutError:
```

```
                "error": {"code": -32603, "message": "Execution timeout."},
            }
        finally:
            self.all_tracked_tasks.pop(active_id, None)

    async def graceful_shutdown(self):
        for task in list(self.all_tracked_tasks.values()):
            task.cancel()
        if self.all_tracked_tasks:
            await asyncio.gather(
                *self.all_tracked_tasks.values(), return_exceptions=True
            )
        self.all_tracked_tasks.clear()
        if self.http_pool:
            await self.http_pool.aclose()
        if self.redis_pool:
            await self.redis_pool.disconnect()
```

4. Behavioral Governance: Operating Modes, Strategies & Scope Classifier

```
from pydantic import BaseModel, Field, DirectoryPath, model_validator
from enum import Enum
from pathlib import Path
import os

class AgentExecutionMode(str, Enum):
    PLAN  = "PLAN"
    AGENT = "AGENT"

class CodeGenerationScope(str, Enum):
    INLINE_SNIPPET      = "INLINE_SNIPPET"
    PATCH_PROPOSAL      = "PATCH_PROPOSAL"
    FILE_MUTATION       = "FILE_MUTATION"
    MULTI_FILE_CHANGESET = "MULTI_FILE_CHANGESET"

class ExecutionStrategy(str, Enum):
    DIRECT          = "DIRECT"
    PLAN_AND_EXECUTE = "PLAN_AND_EXECUTE"
    REACT           = "REACT"
    REFLECTION      = "REFLECTION"

class RigorLevel(str, Enum):
    LOW    = "LOW"
    MEDIUM = "MEDIUM"
    HIGH   = "HIGH"

class ModelTier(str, Enum):
    HAIKU = "HAIKU"
    PRO   = "PRO"

class ToolOperationKind(str, Enum):
    READ             = "READ"
    WRITE            = "WRITE"
    EXTERNAL_MUTATION = "EXTERNAL_MUTATION"

class OptimusToolErrorCode(str, Enum):
    VALIDATION_ERROR = "ERR_TOOL_VALIDATION"
    RESOURCE_ERROR   = "ERR_TOOL_RESOURCE"
    PROVIDER_ERROR   = "ERR_TOOL_PROVIDER"
    TIMEOUT_ERROR    = "ERR_TOOL_TIMEOUT"
```

```
class OptimusToolError(Exception):
    def __init__(self, code: OptimusToolErrorCode, message: str,
                 context: dict = None):
        super().__init__(message)
        self.code    = code
        self.message = message
        self.context = context or {}

class AssumptionItem(BaseModel):
    claim:               str
    basis:               str
    confidence:          str  = "medium"
    requires_verification: bool = True

class AssumptionLedger(BaseModel):
    assumptions: list[AssumptionItem] = Field(default_factory=list)
```

```
class ToolResponse(BaseModel):
    is_success: bool
    payload:    dict = Field(default_factory=dict)
    error_code: Optional[OptimusToolErrorCode] = None
    error_msg:  Optional[str] = None

    def to_jsonrpc_error(self) -> dict:
        code_mapping = {
            OptimusToolErrorCode.VALIDATION_ERROR: -32002,
```

```
            OptimusToolErrorCode.TIMEOUT_ERROR:    -32005,
        }
        rpc_code = code_mapping.get(self.error_code, -32603)
        return {
            "code":    rpc_code,
            "message": f"Tool Execution Failure: {self.error_msg}",
            "data": {
                "tool_error_code": (
                    self.error_code.value if self.error_code else None
                ),
                "context": self.payload,
            },
        }
```

```
def classify_generation_scope(
    line_count: int, file_count: int, creates_or_deletes: bool,
    touches_shared: bool, directory_root_count: int,
) -> CodeGenerationScope:
    if (
        touches_shared
        or directory_root_count > 1
        or (creates_or_deletes and file_count > 1)
        or file_count > 3
    ):
        return CodeGenerationScope.MULTI_FILE_CHANGESET
    if (creates_or_deletes and file_count == 1) or (
        file_count == 1 and line_count > 100
    ):
        return CodeGenerationScope.FILE_MUTATION
    if line_count > 15:
        return CodeGenerationScope.PATCH_PROPOSAL
    return CodeGenerationScope.INLINE_SNIPPET
```

```
def select_strategy(
    task_risk: str, ambiguity: str, mutation_scope: str,
    validation_failed: bool, touches_shared_module: bool,
) -> Tuple[ExecutionStrategy, dict]:
    budget = {
        "max_tool_calls":         5,
        "max_reflection_passes":  0,
        "max_context_tokens":     100000,
        "preferred_model_tier":   ModelTier.HAIKU.value,
    }
    if validation_failed or touches_shared_module:
        budget.update({
            "max_tool_calls":        15,
            "max_reflection_passes":  3,
            "max_context_tokens":    250000,
            "preferred_model_tier":  ModelTier.PRO.value,
        })
        return ExecutionStrategy.REFLECTION, budget
    if task_risk == "HIGH" or mutation_scope == "MULTI_FILE_CHANGESET":
        budget.update({
            "max_tool_calls":        10,
            "max_reflection_passes":  1,
            "max_context_tokens":    150000,
            "preferred_model_tier":  ModelTier.PRO.value,
        })
        return ExecutionStrategy.PLAN_AND_EXECUTE, budget
    if ambiguity == "HIGH":
        budget.update({
            "max_tool_calls":       12,
            "max_context_tokens":  150000,
            "preferred_model_tier": ModelTier.PRO.value,
        })
        return ExecutionStrategy.REACT, budget
    return ExecutionStrategy.DIRECT, budget
```

```
def assert_mutation_allowed(
    mode: AgentExecutionMode, op_kind: ToolOperationKind, operation: str,
):
    if mode == AgentExecutionMode.PLAN and op_kind in {
```

```
        raise OptimusToolError(
            OptimusToolErrorCode.VALIDATION_ERROR,
            f"Access security violation error. "
            f"Mutation blocked in PLAN mode: {operation}",
        )

class MutationGuard:
    def __init__(self, mode: AgentExecutionMode):
        self.mode = mode

    def enforce_gate(self, op_kind: ToolOperationKind, tool_name: str):
        assert_mutation_allowed(self.mode, op_kind, tool_name)
```

```
class ASTOptimizationInput(BaseModel):
    project_path:     DirectoryPath = Field(
        ..., description="Absolute path to project root."
    )
    target_file:      str = Field(
        ..., description="Relative file path string."
    )
    adl_rules:        list[str]          = Field(default_factory=list)
    execution_mode:   AgentExecutionMode  = Field(
        default=AgentExecutionMode.PLAN
    )
    generation_scope: CodeGenerationScope = Field(
        default=CodeGenerationScope.INLINE_SNIPPET
    )
    rigor_budget:     RigorLevel          = Field(default=RigorLevel.LOW)
    assumption_ledger: AssumptionLedger   = Field(
        default_factory=AssumptionLedger
    )

    @model_validator(mode="after")
    def validate_paths_and_safeguards(self) -> "ASTOptimizationInput":
        if Path(self.target_file).is_absolute():
            raise ValueError(
                "Target file must be a relative path configuration."
            )
        try:
            resolved_base = Path(
                os.path.abspath(self.project_path)
            ).resolve(strict=True)
            candidate_path = Path(os.path.abspath(
                os.path.join(str(resolved_base), self.target_file)
            ))
        except (ValueError, FileNotFoundError, RuntimeError):
            raise ValueError(
                "Malformed or invalid project root directory specification."
            )
        try:
            if candidate_path.exists() or candidate_path.is_symlink():
                resolved_target = candidate_path.resolve(strict=False)
                if os.path.commonpath(
                    [str(resolved_base), str(resolved_target)]
                ) != str(resolved_base):
                    raise ValueError(
                        "Breakout attempt detected via target file path alignment."
                    )
            else:
                parent_path     = candidate_path.parent
                resolved_parent = parent_path.resolve(strict=True)
                if os.path.commonpath(
                    [str(resolved_base), str(resolved_parent)]
                ) != str(resolved_base):
                    raise ValueError(
                        "Symlink ancestry breakout attempt detected via target parent folder."
                    )
        except ValueError as ve:
            raise ValueError(
                f"Cross-drive partition boundary constraint or layout failure: {str(ve)}"
            )
        except (FileNotFoundError, RuntimeError):
            raise ValueError(
                "Missing or invalid ancestral directory infrastructure framework structure mapping."
            )
        return self
```

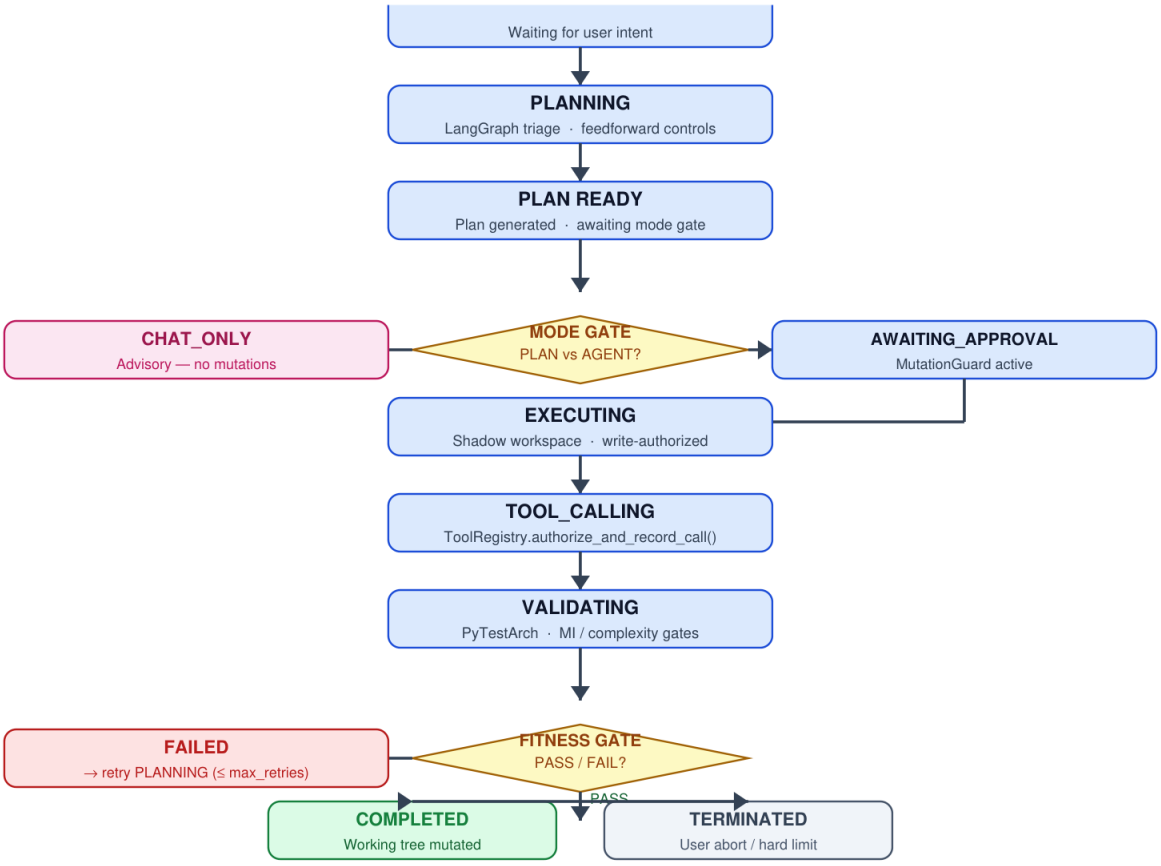
4A. Agent State Model and Execution Pathways

This section defines the complete agent lifecycle — from initial idle state through planning, execution, tool calling, and termination — and specifies which transitions are valid, which are rejected deterministically, and what permissions each mode grants.

Lifecycle State Machine

The agent operates as a deterministic state machine. Every state transition is validated at the control-plane boundary; invalid transitions are rejected with a structured -32002 error before any side-effectful work begins.

Section 4A · Agent Lifecycle State Machine



Deterministic state machine — every transition validated at the mutation boundary (MutationGuard).

Valid and Invalid Transition Table

Every state pair is enumerated below. The harness enforces these deterministically — no model instruction can override a rejected transition.

From	To	Permitted	Condition / Rejection Reason
Idle	Planning	**Yes**	Always — triggered by any user request.
Planning	PlanReady	**Yes**	Requirements clarified; mode selected.
PlanReady	ChatOnly	**Yes**	mode = Plan/Chat; no mutation allowed.
PlanReady	Executing (direct)	**No**	Rejected. Must pass through AwaitingApproval.
PlanReady	AwaitingApproval	**Yes**	mode = Agent; awaits explicit user approval.
AwaitingApproval	Executing	**Yes**	Approval granted within timeout window.
AwaitingApproval	ChatOnly	**Yes**	Approval denied or timeout; falls back advisory.
Executing	ToolCalling	**Yes**	Tool invocation authorised by ToolInvocationPolicy.
Executing	ToolCalling (mutation)	**Cond.**	Mutation tools blocked if mode != Agent.
ToolCalling	Validating	**Yes**	Tool response received; fitness gates run.
Validating	Executing	**Yes**	All gates passed.
Validating	Failed	**Yes**	Any gate fails; retry counter incremented.
Failed	Planning	**Yes**	Retry budget not exhausted (max 3 retries).
Failed	Terminated	**Yes**	Budget exhausted; user escalation emitted.
Executing	Completed	**Yes**	All planned work done; no open gates.
Any	Executing (bypass)	**No**	Rejected -32002. No path bypasses AwaitingApproval.

Mode-Specific Permission Matrix

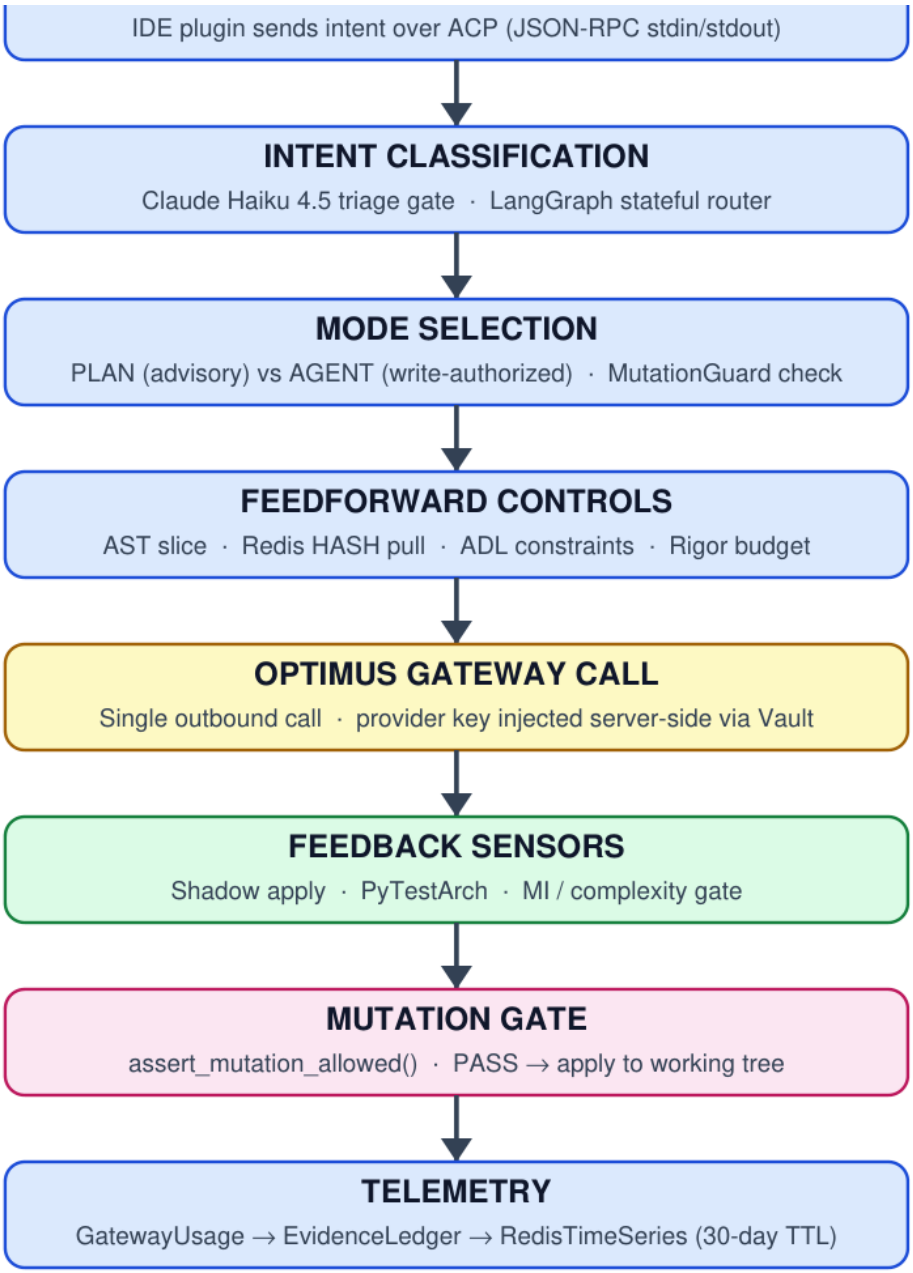
Permissions are enforced at the harness layer, not by model instruction. Any attempt to invoke a mutation tool outside Agent mode is rejected before the tool call is dispatched.

Capability	Plan/Chat	Agent	Enforcement
Read repo / inspect files	**Yes**	**Yes**	Always allowed.
Run read-only analysis tools	**Yes**	**Yes**	Always allowed.
Web search / extract (policy-gated)	**Yes**	**Yes**	ToolInvocationPolicy must approve.
Produce diffs / snippets as text	**Yes**	**Yes**	Returned as text payload, not applied.
Shadow-apply patch (read-only test)	**No**	**Yes**	assert_mutation_allowed() gate.
Mutate working tree (write files)	**No**	**Yes**	assert_mutation_allowed() + MutationGuard.
Run shell / build / test	**No**	**Yes**	Agent mode + composite fitness gate.
Append cost / audit telemetry	**Yes**	**Yes**	Always-on; append-only.
Modify telemetry / ledger entries	**No**	**No**	Immutable after write.

Execution Pathway

The execution pathway shows how a user request flows through intent classification, mode selection, mutation gating, and telemetry — with the Optimus Gateway as the single external call point.

Section 4A · Execution Pathway



End-to-end execution pathway. Providers (GLM / Haiku / Sonnet / Opus) are selected by the Gateway via model alias.

Execution Strategy Selection

The agent selects one of four execution strategies based on task ambiguity, mutation risk, and prior validation outcomes. Strategy selection is harness-controlled, not model-controlled.

Strategy	Trigger	Behaviour	Cost Profile
Direct Execution	LOW rigor, no mutations	Single model call; apply result; no reflection.	<=5 tool calls, 0 reflections, Haiku.
Plan-and-Execute	MEDIUM rigor, mutation required	Structured plan; execute step-by-step; validate each step.	<=10 tool calls, 1 reflection, Pro.
ReAct	HIGH rigor, ambiguous/multi-file	Interleave reasoning and tool calls; update plan per observation.	<=15 tool calls, up to 3 reflections, Pro.
Reflection	Exception: gate failure or shared-core	Triggered by failed fitness gate or MULTI_FILE_CHANGESET. Not a default loop.	Bounded to 3 passes max.

Tool Invocation Decision Rules

Tool invocation is always harness-controlled. The model may express intent to call a tool; the ToolInvocationPolicy decides whether it is permitted. Rules are evaluated in priority order; first match wins.

```
# ToolInvocationPolicy evaluation order (first match wins)

if mode == PLAN_CHAT and tool.is_mutation:
    REJECT  # -32002: mutation tools forbidden in Plan/Chat mode

if tool.class == WEB_SEARCH:
    if not any([user_requested_external_fact, task_mentions_unknown_api,
                task_touches_security_cves]):
        REJECT  # local evidence first; no casual web calls
    require reason_code in VALID_REASON_CODES
    ALLOW with EvidenceRequest(query=verbatim, reason=reason_code)

if tool.class == WEB_EXTRACT:
    require url in prior_search_result_set  # provenance check
    ALLOW with EvidenceExtractRequest(url=url)

if tool.class == MUTATION and not approval_granted:
    REJECT  # -32002: no mutation before AwaitingApproval clears

DEFAULT: ALLOW  # local read/inspect/analysis tools
```

Failure and Retry Lifecycle

When a fitness gate fails, the agent records the failure signal, increments the retry counter, and re-enters Planning with failure context injected into the next prompt. After three consecutive failures, execution terminates and the user is escalated with a structured failure report.

```
class RetryPolicy:
    max_retries: int = 3
    backoff_base_ms: int = 500
    retry_on: tuple = (TransientGatewayError, ProviderRateLimitError)
    abort_on: tuple = (PermanentGatewayError, PolicyViolationError,
                       BudgetExhaustedError)

    def on_gate_failure(self, failure: FitnessGateFailure) -> RetryAction:
        self.failure_log.append(failure)
        if self.retry_count >= self.max_retries:
            return RetryAction.ESCALATE_TO_USER
        if failure.is_permanent:
            return RetryAction.ABORT_WITH_REPORT
        self.retry_count += 1
        return RetryAction.REPLAN_WITH_FAILURE_CONTEXT
```

5. ADL Parser Continuity & Architectural Evolution

```
from abc import ABC, abstractmethod

class ADLParserStrategy(ABC):
    @abstractmethod
    def parse_and_validate(
        self, rules: list, patch_diff: str
    ) -> Tuple[bool, list]:
        pass

class PrototypeRegexADLParser(ADLParserStrategy):
    def __init__(self):
        # Aligned with single escapes inside a raw string
        # configuration to match input streams correctly.
        self.dep_pattern = re.compile(
            r"ASSERT\((?P<la>\w+)\s+HAS\s+NO\s+DEPENDENCY"
            r"\s+ON\s+(?P<lb>\w+)\)"
        )

    def parse_and_validate(
        self, rules: list, patch_diff: str
    ) -> Tuple[bool, list]:
        violations = []
        for rule in rules:
            match = self.dep_pattern.match(rule)
            if not match:
                continue
            la, lb   = match.group("la"), match.group("lb")
            lb_lower = lb.lower()
            has_import     = f"import {lb_lower}" in patch_diff
            has_spring_dep = (
                f"org.springframework.{lb_lower}" in patch_diff
            )
            if has_import or has_spring_dep:
                violations.append({
                    "rule":   rule,
                    "metric": "Structural Pollution",
                    "value":  f"Violation: {la}->{lb}",
                })
        return len(violations) == 0, violations
```

6. Resilient Aggregator Calling Layer and Tenacity Rules

All model completions route from the agent to the strict-loopback Gateway using the local shared secret. The Gateway resolves an Optimus model alias to an aggregator model identifier and calls the approved OpenAI-compatible upstream: OpenRouter by default, or Vercel AI Gateway if retained.

The aggregator credential exists only in Gateway-owned local configuration or OS credential storage. There is no Vault, hosted Optimus account, or direct single-provider adapter.

Two agent-facing shapes, one upstream transport

- `/v1/responses` accepts the Responses API `input` field.
- `/v1/chat/completions` accepts the Chat Completions `messages` array.
- The validator rejects `messages` at `/v1/responses` and `input` at `/v1/chat/completions`.
- Both paths converge on one authenticated completion service.
- The surviving transport follows `UrllibOpenAICompatibleClient` in `src/optimus_gateway/upstream_client.py`.

Transient network, timeout, rate-limit, and provider-availability failures may retry. Permanent authentication, schema, policy, and malformed-usage failures do not retry. A transient call has at most three attempts; every retry records attempt number, classification, latency, and disposition.

Provider-reported `usage.cost` is authoritative in the corrected architecture. Missing, null, negative, or malformed usage/cost fails closed before generated output is accepted.

6.1 OpenAI-compatible upstream pattern

```
class UrllibOpenAICompatibleClient:
    """Gateway-owned aggregator transport.

    The agent never receives the upstream URL or credential. The transport
    has no direct-provider selection branch.
    """

    def __init__(self, *, api_key: str, base_url: str) -> None:
        self._api_key = api_key
        self._base_url = base_url.rstrip("/")

    def create_message(
        self, *, model: str, input_text: str
    ) -> ProviderMessageResult:
        payload = {
            "model": model,
            "messages": [{"role": "user", "content": input_text}],
        }
        body = self._post_json("/chat/completions", payload)
        return parse_openai_compatible_message_and_usage(body)
```

`parse_openai_compatible_message_and_usage` must validate output, provider request identity, billing units, token/cache detail, resolved model/version when present, and provider-reported USD cost. The Gateway returns the requested agent-facing shape plus the same validated GatewayUsage contract.

```
Agent /v1/responses "input" --------\
                                    > completion service
Agent /v1/chat/completions "messages"/       |
                                             v
                        OpenAI-compatible aggregator transport
```

The upstream transport may use Chat Completions while the Gateway preserves both agent-facing contracts. Those shapes must never be mixed.

7. Composite Fitness Engine Scaffolding

```
class FitnessGateInterface(ABC):
    @abstractmethod
    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        pass

class DependencyRuleGate(FitnessGateInterface):
    def __init__(self, parser: ADLParserStrategy):
        self.parser = parser

    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        passed, violations = self.parser.parse_and_validate(
            state.get("adl_rules", []), state.get("generated_patch", "")
        )
        return passed, {"violations": violations}

class PlaceholderMetricGate(FitnessGateInterface):
    def __init__(self, max_complexity: int = 15):
        self.max_complexity = max_complexity

    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        return True, {
            "metrics": {
                "cyclomatic_complexity":  4,
                "maintainability_index": 82,
            }
        }

class PlaceholderTestArchGate(FitnessGateInterface):
    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        return True, {
            "testarch_status": "COMPLIANT",
            "graph_cyclical":  False,
        }

class CompositeFitnessEngine:
    def __init__(self, gates: list[FitnessGateInterface]):
        self.gates = gates

    def execute_harness_check(self, state: dict) -> dict:
        combined_telemetry = {}
        for gate in self.gates:
            passed, telemetry_delta = gate.evaluate_gate(state)
            combined_telemetry.update(telemetry_delta)
            if not passed:
                return {
                    "fitness_verdict": "FAIL",
                    "telemetry":       combined_telemetry,
                }
        return {
            "fitness_verdict": "PASS",
            "telemetry":       combined_telemetry,
        }
```

8. Patch Workspace Lifecycle Boundaries

Applying patches exposes systems to directory traversal attacks. Optimus normalises boundaries using strict commonpath comparisons and strictly isolates validation steps entirely inside a localised shadow workspace environment.

- **Shadow Workspace Environment**: Patches are written exclusively to a workspace-local shadow directory located at .optimus/shadow_env/. The patch diff is programmatically parsed, applied, and validated against the duplicate code tree via python file mutation loops.
- **Operating Mode Enforcement Contra**ct: If execution_mode equals AgentExecutionMode.PLAN, tool operations matching ToolOperationKind.WRITE or ToolOperationKind.EXTERNAL_MUTATION are completely bypassed and blocked from modifying local structures. Read-only inspection tools remain fully authorised.
- **Atomic Apply to Working Tree & Surfacing Rejec**tions: Modifications are applied to the working tree only after a clean PASS verdict, guaranteeing transaction atomicity and immediate rollback safety on any system fault. If a gate trips a FAIL signature, the patch is rejected, the transaction aborts, and the duplicate shadow environment is cleanly purged.

9. Tool Registry, Invocation Policy & Evidence Ledger

Tool use is policy-driven first, model-requested second: the model may request evidence, but the harness alone decides whether a tool is permitted, necessary, and cost-justified for the active execution mode and rigor tier. This section defines the typed tool registry, the deterministic invocation policy matrix, and the Evidence Ledger schema that records every external lookup alongside the Assumption Ledger.

A. Tool Class Enum & Registry Contract

```
class ToolClass(str, Enum):
    LOCAL_REPO_DISCOVERY         = "LOCAL_REPO_DISCOVERY"
    VALIDATION_GATE              = "VALIDATION_GATE"
    PATCH_WORKSPACE              = "PATCH_WORKSPACE"
    COST_TELEMETRY               = "COST_TELEMETRY"
    WEB_EVIDENCE                 = "WEB_EVIDENCE"
    PACKAGE_AND_ADVISORY_METADATA = "PACKAGE_AND_ADVISORY_METADATA"
    EXTERNAL_EXECUTION           = "EXTERNAL_EXECUTION"

class EvidenceReasonCode(str, Enum):
    API_DOCS_OUTDATED = "API_DOCS_OUTDATED"
    CURRENT_FACT      = "CURRENT_FACT"
    SECURITY_ADVISORY = "SECURITY_ADVISORY"
    USER_REQUESTED    = "USER_REQUESTED"
    PACKAGE_VERSION   = "PACKAGE_VERSION"

class SearchDepth(str, Enum):
    """Internal rigor presets. NOT assumed to equal the gateway's wire
    values 1:1 -- see TAVILY_SEARCH_DEPTH_MAP in section C, which is
    the only place an internal value is translated into the string
    actually sent to the provider via the gateway."""
    BASIC    = "basic"
    FAST     = "fast"
    ADVANCED = "advanced"

class ToolRegistryEntry(BaseModel):
    tool_name:             str
    tool_class:            ToolClass
    allowed_modes:         list[AgentExecutionMode]
    requires_policy_trigger: bool = False
    is_mutating:           bool = False
    max_calls_per_run:     Optional[int] = None

class ToolRegistry(BaseModel):
    entries:     dict[str, ToolRegistryEntry] = Field(default_factory=dict)
    # In-process call counters, keyed by f"{run_id}:{tool_name}". Single
    # process / single-instance only -- see the Redis-backed variant
    # below for concurrent or multi-process execution.
    call_counts: dict[str, int] = Field(default_factory=dict)

    def check_static_policy(
        self, entry: Optional[ToolRegistryEntry],
        mode: AgentExecutionMode, policy_triggered: bool,
    ) -> bool:
        if entry is None:
            return False
        if mode not in entry.allowed_modes:
            return False
        if entry.requires_policy_trigger and not policy_triggered:
            return False
        return True
```

```
    def is_allowed(
        self, tool_name: str, mode: AgentExecutionMode,
        policy_triggered: bool, run_id: str,
    ) -> bool:
        """READ-ONLY inspection (e.g. UI previews, dry-run explainers).
        Does not increment the call counter. Never use this to gate an
        actual tool invocation -- use authorize_and_record_call() below,
```

```
        entry = self.entries.get(tool_name)
        if not self.check_static_policy(entry, mode, policy_triggered):
            return False
        if entry.max_calls_per_run is not None:
            counter_key = f"{run_id}:{tool_name}"
            calls_so_far = self.call_counts.get(counter_key, 0)
            if calls_so_far >= entry.max_calls_per_run:
                return False
        return True

    def authorize_and_record_call(
        self, tool_name: str, mode: AgentExecutionMode,
        policy_triggered: bool, run_id: str,
    ) -> bool:
        """The only method that should gate an actual tool call. Checks
        the static policy and the call-count ceiling, then increments
        the counter, as a single atomic step -- so authorization and
        bookkeeping can never drift apart, and a caller cannot forget
        to record a call it was just allowed to make."""
        entry = self.entries.get(tool_name)
        if not self.check_static_policy(entry, mode, policy_triggered):
            return False
        counter_key = f"{run_id}:{tool_name}"
        if entry.max_calls_per_run is not None:
            calls_so_far = self.call_counts.get(counter_key, 0)
            if calls_so_far >= entry.max_calls_per_run:
                return False
        self.call_counts[counter_key] = (
            self.call_counts.get(counter_key, 0) + 1
        )
        return True
```

The in-process dict above is sufficient for Phase 1's single-instance daemon, but is not safe across multiple worker processes and does not survive a restart mid-run. The production-grade path swaps the in-memory counter for a Redis-backed atomic increment, keyed identically and protected by a TTL matching the run's expected lifetime:

```
async def authorize_and_record_call_redis(
    registry: ToolRegistry, redis_client: aioredis.Redis,
    tool_name: str, mode: AgentExecutionMode,
    policy_triggered: bool, run_id: str, run_ttl_secs: int = 3600,
) -> bool:
    entry = registry.entries.get(tool_name)
    if not registry.check_static_policy(entry, mode, policy_triggered):
        return False
    if entry.max_calls_per_run is None:
        return True
    counter_key = f"toolcalls:{run_id}:{tool_name}"
    # INCR is atomic at the Redis level, so concurrent callers cannot
    # race past the ceiling the way two in-process checks could.
    new_count = await redis_client.incr(counter_key)
    if new_count == 1:
        await redis_client.expire(counter_key, run_ttl_secs)
    if new_count > entry.max_calls_per_run:
        await redis_client.decr(counter_key)
        return False
    return True
```

B. Deterministic Invocation Policy Matrix

The policy matrix maps task signals to permitted tool classes. The model never selects a tool class directly; it raises a typed signal and the harness resolves the matching policy row. PACKAGE_AND_ADVISORY_METADATA deliberately covers both package-registry lookups (PyPI, npm, Maven Central) and security-advisory lookups (OSV.dev, GitHub advisories): both are read-only, policy-triggered, and keyed by a package or CVE identifier, so a single tool class keeps the registry simple while the signal name (TASK_TOUCHES_DEPENDENCIES vs. TASK_TOUCHES_SECURITY_CVE) still makes the routing intent explicit.

9C. Typed Evidence Acquisition Wrappers

The agent authenticates only to the loopback Gateway. Search-provider mechanics remain behind `/v1/tools/web/search`; extract, package, and advisory capabilities have independent dependencies.

Strict-loopback settings

```
class OptimusGatewaySettings(BaseModel):
    gateway_url: str = "http://127.0.0.1:8765"
    optimus_api_key: SecretStr

    @model_validator(mode="after")
    def validate_loopback_gateway(self) -> "OptimusGatewaySettings":
        parsed = urlsplit(self.gateway_url)
        if parsed.scheme not in {"http", "https"}:
            raise ValueError("Gateway URL must use HTTP(S)")
        if parsed.username is not None or parsed.password is not None:
            raise ValueError("Gateway URL must not contain userinfo")
        if parsed.hostname not in {"127.0.0.1", "localhost", "::1"}:
            raise ValueError("Gateway URL must resolve to strict loopback")
        return self
```

There is no production mode, built-in hosted origin, extra-origin override, signed tenant profile, or non-loopback trust seam. The final source must remain aligned with the separately reviewed strict-loopback implementation.

9C.1 Deterministic search contract

The Gateway converts an authorized search request into a dedicated OpenRouter Chat Completions request using one cheap Flash/Haiku-class model, low `max_tokens`, and `plugins:[{"id":"web"}]`. Search is never attached to the primary generation call.

```
{
  "model": "google/gemini-2.5-flash-lite",
  "messages": [{"role": "user", "content": "<minimal search request>"}],
  "max_tokens": 16,
  "plugins": [{
    "id": "web",
    "max_results": 3,
    "search_prompt": "<policy-bounded query>"
  }]
}
```

The Gateway forwards the effective domain policy, parses only `annotations[].url_citation`, independently revalidates every returned URL, and copies upstream usage/cost. Assistant prose and annotation character offsets are not evidence.

Measured acceptance baseline

3/3 minimal-call annotations present	**0** include-domain violations
0 exclude-domain violations	**9/9** citations structurally complete
7,068.7 ms mean latency per search	**$0.0051584** mean reported cost

Three to five sequential searches project to 21.2-35.3 seconds. The supplied Tavily comparison is about 1-2 seconds and $0.008/search; it was not measured in this spike.

9C.2 Plugin deprecation and successor gate

The deterministic OpenRouter plugin is deprecated. A live release probe must block release on removal or behavior change.

The first successor evaluation is a dedicated server-tool request with:

- `max_uses: 1`;
- a one-call budget;
- mandatory search-use accounting;
- valid structured annotations;
- domain-policy enforcement;
- fail-closed behavior when execution evidence is absent.

This is verified-or-fail, not deterministic by construction. If it fails, the fallback is a standalone search provider with a second credential and balance.

9C.3 Bounded direct extract

Extract performs a Gateway-side HTTPS fetch only for URLs recorded by a prior approved search in the same run. It:

- revalidates every redirect and final host;
- rejects URI userinfo and private, link-local, or loopback resolution;
- bounds connect/read time, bytes, redirects, and extracted characters;
- accepts only supported textual media;
- treats HTML and extracted text as untrusted input;
- never executes page content.

The spike fetched 270,008 bytes and parsed 60,077 characters in 589.5 ms. A 1,000,000-byte limit failed closed on a legitimate larger page; production limits and streaming behavior require explicit tests.

9C.4 Independent tool dependency construction

Tavily currently gates all four typed tools because `build_tool_dependencies()` returns `None` when `TAVILY_API_KEY` is absent. That coupling must be removed regardless of the selected search backend.

```
@dataclass(frozen=True)
class ToolDependencies:
    search: SearchBackend | None
    extract: ExtractBackend
    package: PackageRegistryBackend
    advisory: AdvisoryBackend


def build_tool_dependencies(settings: GatewaySettings) -> ToolDependencies:
    return ToolDependencies(
        search=build_search_backend_if_configured(settings),
        extract=BoundedHttpExtract(...),
        package=PublicPackageRegistries(...),
        advisory=OsvAdvisoryClient(...),
    )
```

Route registration is capability-specific:

Capability	Availability rule
Search	Selected backend configured and live acceptance gate passes
Extract	Bounded HTTPS fetch and prior-search provenance support available
Package lookup	Always available with public PyPI/npm/Maven clients
Security advisory	Always available with public OSV client

Do not delete the Tavily adapter until replacement acceptance tests and rollback review pass. Record that a standalone search backend may need to return if the aggregator search path is withdrawn.

9D. Gateway Server-Side Policy Revalidation

The Gateway independently revalidates every privileged input. Agent-side checks are defense in depth and never authoritative.

Control	Gateway enforcement
Allowed domains	Intersect caller request with local allowlist; forward effective policy; reject returned URLs outside it
Extract provenance	Require exact prior search result for the same `run_id`; revalidate redirects and final URL
Budget	Enforce current-run USD cap against Gateway ledger before billable dispatch
Call caps	Key by run and tool; do not share a paid-search configuration gate
Tool policy	Recheck tool class, policy signal, execution mode, and authenticated local subject
Usage	Reject missing, malformed, negative, or unparseable provider-reported cost

```
        def authorize_tool_call(request, *, policy, ledger):
            effective_domains = policy.intersect_domains(request.domains)
            policy.require_tool_allowed(request.tool, request.execution_mode)
            policy.require_call_capacity(request.run_id, request.tool)
            ledger.require_budget_capacity(request.run_id, request.max_cost_usd)
            return AuthorizedToolCall(
                request=request,
                effective_domains=effective_domains,
            )
```

There is no org/project dimension. Cross-run spend policy remains `P9.85-FU-3` and is not added by this correction.

9E. Evidence Ledger Schema

Evidence entries retain structural provenance and provider-reported usage without adding a second normalized-credit charge.

```python
class EvidenceLedgerEntry(BaseModel):
    evidence_id: str
    run_id: str
    session_id: str
    request_id: str
    gateway_request_id: str
    provider_request_id: str | None
    provider: str
    model: str | None
    model_version: str | None
    cache_hit: bool
    billing_units: int
    cost_usd: Decimal
    source_url: AnyHttpUrl | None
    trust: Literal["trusted", "untrusted"]
    policy_reason: str
    recorded_at: datetime
```

`cost_usd` is copied from validated GatewayUsage. `billing_units` remains provider-native accounting normalized only to a stable integer field. Package/advisory operations may record zero billable units without inventing vendor charges.

Reconciliation methods are `total_cost_usd()` and `total_billing_units()`. Credit-named fields and `total_credits()` are removed under the separate USD rename.

9E.1 Protocol-visible USD rename custody

`ledger_run_total_credits` is exposed in ACP result payloads for search, extract, package, and advisory responses. Its rename is therefore a wire-contract migration, not an internal refactor.

The separate subtask must:

1. define the USD-named response field;
2. decide and document any compatibility interval;
3. update ACP schemas and independently authored ACP-client expectations;
4. update unit, integration, golden, and real-client evidence;
5. prove the old credit-named response field is retired;
6. preserve the field's existing USD semantics;
7. add no cross-run policy.

Record	Required evidence
Search/extract response	GatewayUsage and run-total USD reconcile
Package/advisory response	Free operation remains non-fabricated and run total is stable
EvidenceLedger	Decimal-safe sum by run/session/request identity
ACP schema	USD field documented and compatibility decision explicit
Real ACP client	Response parses without project-authored client assumptions

The ledger remains append-only and must reject a duplicate charge for the same `gateway_request_id`.

10. Usage Accounting Service and TimeSeries Policy

`UsageAccountingService` accepts only validated GatewayUsage/ProviderUsage records. It persists provider-reported billing units, tokens/cache details, model/version, and `cost_usd`.

```
        def record_usage(self, usage: GatewayUsage, *, context: RequestContext) -> None:
            require_nonempty(usage.gateway_request_id)
            require_nonnegative_decimal(usage.cost_usd)
            require_nonnegative_int(usage.billing_units)
            self.ledger.append(
                ProviderUsage.from_gateway_usage(usage, context=context)
            )
```

It never substitutes a locally calculated charge when provider cost exists. Missing, null, negative, or malformed cost fails closed before generated content or evidence is applied.

A versioned price snapshot may remain as labeled diagnostic/comparison metadata only. It cannot overwrite provider-reported cost or silently supply a release-grade value.

Required identity

- `run_id`
- `session_id`
- `request_id`
- `gateway_request_id`
- `provider_request_id` when returned
- aggregator and actual provider when returned
- requested alias and resolved model/version

10.1 RedisTimeSeries persistence

RedisTimeSeries retention and idempotent creation mechanics remain unchanged.

```
TS.CREATE optimus:usage:<run_id>:cost_usd
  RETENTION 2592000000
  LABELS run_id <run_id> metric cost_usd

TS.CREATE optimus:usage:<run_id>:billing_units
  RETENTION 2592000000
  LABELS run_id <run_id> metric billing_units
```

Use `TS.ALTER` when an existing series needs required labels or retention correction. Never drop existing measurements during schema alignment.

Reconciliation

For each accepted Gateway request:

```
Gateway response cost_usd
  == ProviderUsage.cost_usd
  == EvidenceLedger entry cost_usd, when evidence-producing
  == RedisTimeSeries appended value
```

Duplicate `gateway_request_id` records must be idempotent or rejected before a second charge is written. Price snapshots are diagnostic only. Observability export is not a billable usage record.

10A. Provider Usage Ledger and Observability Export

ProviderUsage persists the wire-level GatewayUsage fields verbatim and adds reconciliation attribution: aggregator, actual provider, resolved model/version, service, native unit, run/session/request IDs, and an optional diagnostic price-snapshot ID.

It does not add an Optimus credit charge. `cost_usd` is the upstream-reported aggregator charge. Public package/advisory calls record operational evidence without invented cost.

OpenTelemetry export

The agent sends authenticated structured trace events to `/v1/observability/traces`. The Gateway:

1. validates the event schema and local subject;
2. redacts credentials and sensitive content;
3. maps fields to OTel spans/events;
4. exports OTLP;
5. records delivery status and trace identifiers.

Phoenix is the documented local default. Required attributes include run, session, request, Gateway/provider request, execution mode, generation scope, model/provider, cache, `cost_usd`, billing units, policy/tool, validation, retry, and failure fields.

Trace delivery has no allocated or amortized per-request observability charge.

11. Sprint 1 Implementation Checklist

Trust boundary

- Accept `127.0.0.1`, `localhost`, and `::1`; reject every non-loopback Gateway URL.
- Remove production-mode, extra-origin, hosted-origin, and tenant-profile bypasses.
- Prove WSL2 agent and Gateway run in the same network namespace.
- Prove the agent process resolves only Gateway URL/shared secret.
- Prove the Gateway credential never appears in agent state, logs, errors, telemetry, or children.

Model transport

- Use OpenRouter as default aggregator and the OpenAI-compatible transport.
- Retain Vercel only if Python integration is modest; otherwise assign a named backlog entry.
- Remove direct provider branches and `anthropic_client.py`.
- Parse provider-reported usage/cost; fail closed when incomplete.
- Bound transient attempts at three and classify permanent failures.

Tool capability

- Separate package/OSV route construction from paid-search configuration.
- Keep Tavily until replacement acceptance and rollback review pass.
- Prove deterministic annotations, domains, URL revalidation, usage, and plugin live gate.
- Prove extract redirect, SSRF, media, size, time, and provenance controls.

11.1 Accounting, telemetry, and release evidence

Accounting

- Persist provider-reported Decimal-safe `cost_usd` and billing units.
- Preserve run/session/request/Gateway/provider request attribution.
- Remove credit-named fields in the separate wire-aware USD migration.
- Prove RedisTimeSeries retention and idempotent creation/alter behavior.

Telemetry

- Export real OTLP spans to a live Phoenix evidence tier.
- Prove no LangSmith dependency, key, or direct backend egress.
- Record validation, retry, tool-policy, and final-disposition attributes.

Release gate

The agent process has only Gateway URL/shared secret. The Gateway process has only the approved aggregator key after migration acceptance. No direct-provider, Tavily, or LangSmith key remains.

Real evidence tiers must use their named dependencies. ACP protocol evidence uses the independent `acpx` client. Fake-based tests remain unit evidence and cannot justify live sign-off.

The deterministic search plugin is release-blocking while it remains the Phase 1 path. A successor server tool is not accepted until live verified-or-fail evidence proves exactly one search, search-use accounting, valid annotations, domain controls, and fail-closed behavior.

11A. Test Coverage and Observability Cross-Reference

The Test Strategy v1.5 is authoritative for coverage, release, live-dependency, and trace evidence.

Code coverage remains at least 80% aggregate production-code coverage, with safety-critical modules protected from regression. OTel/OTLP trace assertions validate debugging and regression fields but do not count toward code coverage.

Evidence class	Required dependency
Unit	Fakes allowed; no network/I/O unless intrinsic
Redis integration	Real TimeSeries-capable Redis
Gateway live	Real Optimus credentials and live local Gateway
ACP protocol	Independent `acpx` client
Trace live	Real OTLP export to Phoenix
Release	Agent/Gateway credential and egress scopes proven separately

LangSmith trace assertions and amortized observability accounting are deleted.

12. Guardrail and Workflow Component Contracts

All model-touching guardrails use the same loopback Gateway, developer aggregator account, USD budget, provider-reported cost, and OTel trace path.

`LoopBudgetPolicy.max_budget_credits` becomes `max_budget_usd` in the separate field-migration subtask. The rename adds no cross-run policy.

Verified-or-fail server-tool successor

A server-tool successor is not accepted until a dedicated live request:

- declares `max_uses: 1`;
- performs exactly one search;
- reports search-use accounting;
- returns structurally valid annotations;
- enforces include/exclude domains;
- fails closed when search execution evidence is absent.

MCP disposition

MCP is outside this correction. No Gateway route, endpoint, tool contract, or diagram branch is added. `P11-FU-3` remains open and `P11-FEAT-GATEWAY-MCP` remains blocked.

12D. Curated Workflow Skills

Curated skills remain on-demand procedural artifacts governed by the same permission and trust boundaries as every other agent capability.

- Skills are curated, reviewed, versioned, and narrow.
- A skill cannot widen the agent’s tool surface or override project/user deny rules.
- Generated skills remain draft-only until reviewed and promoted.
- Skill resolution is deterministic and carries no model-call cost.
- Completion evaluation uses a cheap model through the same loopback Gateway.
- The completion evaluator is governed by `max_budget_usd`, provider-reported cost, and the same OTel/OTLP trace path as every other model call.

Document closure

This LLD documents a local-first Gateway with one agent-facing shared secret, one developer-controlled aggregator credential, independent typed tools, provider-reported cost, and vendor-neutral observability.

No hosted Optimus service, tenant wallet, Vault, direct provider adapter, LangSmith dependency, or MCP Gateway endpoint is part of Phase 1.